

COMP9444: Neural Networks and Deep Learning

COMP9444_ Group Project Report

Ashwin Sudhir Jamgade
z5392534

Ishu Rajput
z5611182

Shashwat Pasari
z5623301

Xiangyu Dou
z5607625

Abstract—This project develops a traffic sign recognition system for autonomous driving using computer vision. We compare three models: a simple MLP baseline, a custom Convolutional Neural Network (CNN) for real-time use, and a fine-tuned ResNet-50 for high accuracy. Unlike traditional methods such as SVM with LBP/SIFT features, CNNs and ResNets learn features directly from images, removing the need for manual feature engineering. Using the GTSRB dataset, we apply normalization and targeted augmentation to improve robustness under varying lighting, scale, and perspective. Results show that ResNet-50 achieves the highest accuracy, while the custom CNN offers an optimal balance between speed and accuracy for deployment.

Index Terms—traffic sign recognition, GTSRB, deep learning, CNN, transfer learning

I. INTRODUCTION

Advanced Driver Assistance Systems (ADAS) and autonomous vehicles rely on fast and reliable perception to ensure safety and compliance with traffic regulations. Traffic signs convey legally binding information and safety-critical alerts; misinterpretation can lead to unsafe driving behavior. Accurate traffic sign recognition (TSR) is therefore essential for enabling vehicles to make informed decisions in real time.

Traditional TSR pipelines have used hand-crafted features such as Local Binary Patterns (LBP) or Scale-Invariant Feature Transform (SIFT) with classifiers like Support Vector Machines (SVM). While interpretable, these approaches require manual feature engineering and degrade under real-world variability. Deep learning methods, in contrast, learn hierarchical features directly from raw pixels, improving robustness to environmental changes.

In this project we therefore focus on end-to-end learning with three models, MLP (linear baseline): A simple reference to show the value of learned spatial features. Custom CNN: Compact, fast, and deployment-friendly, it learns localized filters suited to traffic signs and aims for real-time inference. ResNet-50 (transfer learning): A deeper model pre-trained on large image datasets, fine-tuning helps capture richer features and improves robustness.

II. RELATED WORK

Traffic sign recognition has progressed from early hand-crafted feature methods to modern deep learning architectures. Early approaches employed descriptors such as Histogram of Oriented Gradients (HOG) and colour histograms, followed by classifiers like Support Vector Machines (SVMs) [1]. These methods performed adequately under controlled conditions but

struggled with variations in lighting, motion blur, and partial occlusion.

The advent of deep Convolutional Neural Networks (CNNs) revolutionised computer vision [8], enabling end-to-end learning of hierarchical features from raw images. On GTSRB, Cireřan et al. [5] achieved over 99% accuracy using multi-column deep networks, establishing CNNs as the dominant approach.

Residual Networks (ResNets), introduced by He et al. [2], addressed the vanishing gradient problem via skip connections, enabling the effective training of very deep models. ResNet-50, a 50-layer variant, balances accuracy and computational efficiency and has shown strong performance in both natural and remote sensing imagery.

Beyond ResNet, modern architectures have also been applied to GTSRB. DenseNet [6] improves feature propagation through dense connectivity, while the Vision Transformer (ViT) [7] models long-range dependencies using self-attention mechanisms. Studies such as Kundu et al. (2022) [10] have shown that transformer-based or hybrid CNN-transformer models can rival or surpass traditional CNNs on GTSRB when pre-trained on large datasets.

Recent work has explored attention modules, generative augmentation for minority classes [9], and lightweight networks such as MobileNet for real-time embedded deployment. These developments highlight a trend towards architectures that maintain high accuracy while meeting the efficiency constraints of ADAS systems.

Our work builds on these advances by systematically comparing a custom CNN, a transfer-learned ResNet-50, and a baseline MLP within a consistent training and evaluation framework.

III. DATASET AND PREPROCESSING

Dataset. GTSRB [3] contains 51,839 labelled images across 43 classes (39,209 for training, 12,630 for testing). The dataset exhibits substantial intra-class variation (viewpoint, weather, partial occlusion) and a long-tailed class distribution.

Splits & leakage control. We fix random seeds and partition the official training set into an 80/20 train/validation split *before* any balancing or augmentation. Class-balanced sampling and targeted augmentation are applied to the *training* split only, validation and test splits remain untouched to avoid data leakage.

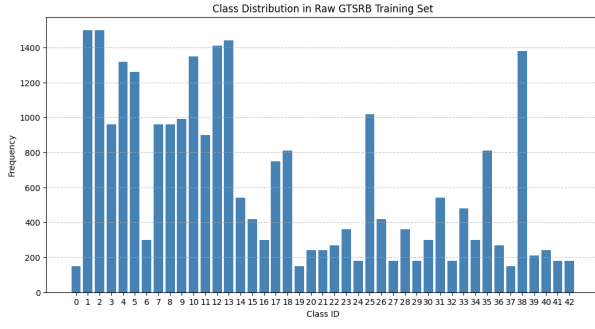


Fig. 1. Class distribution in the raw GTSRB training set (illustrating imbalance).

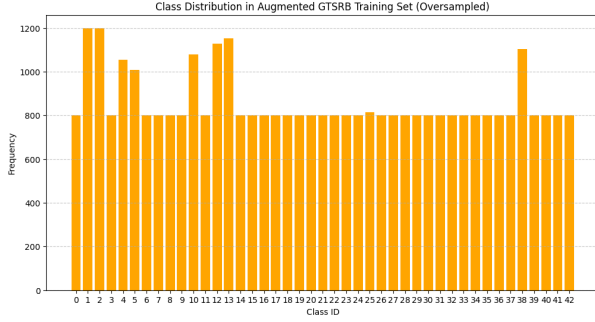


Fig. 2. Class distribution of the augmented GTSRB training set after balancing with oversampling.

Normalisation and resizing. For the CNN and MLP, inputs are resized to 64×64 and normalised using dataset statistics (mean = $[0.3403, 0.3121, 0.3214]$, std = $[0.2724, 0.2608, 0.2669]$). For transfer learning, the ResNet-50 branch uses 224×224 and ImageNet normalisation.

Imbalance handling. To increase minority class coverage, we use random rotation/affine transforms, translation, and colour jitter, together with class-aware sampling during training.

IV. METHODOLOGY

Our project’s approach to traffic sign recognition was multifaceted, deliberately structured to compare a custom-designed neural network against both an advanced transfer learning model and a foundational baseline. We carefully selected and implemented three distinct models—a custom Convolutional Neural Network (CNN), a pre-trained ResNet-50, and a simple linear classifier—to rigorously evaluate the trade-offs in performance, complexity, and efficiency. The following sections provide a detailed look into the design, training, and rationale behind each of these models.

A. Custom Convolutional Neural Network (CNN)

We designed a custom CNN from the ground up, with the aim of creating a model that was both highly accurate and computationally efficient for our specific task. The architecture was built as a sequential stack of layers, designed to progressively extract a rich hierarchy of features from the

input images. This process starts by identifying simple patterns like edges and colors in the initial layers, and gradually builds to recognizing the more complex shapes and emblems characteristic of traffic signs.

The core of our architecture consists of three convolutional blocks. Each block is structured with two convolutional layers, each using a 3×3 kernel to capture spatial features. Following these layers, we applied Batch Normalization to stabilize training and a Rectified Linear Unit (ReLU) activation function to introduce non-linearity. A 2×2 MaxPooling layer with a stride of 2 was then used to downsample the feature maps, effectively reducing the spatial dimensions and increasing the receptive field. To mitigate the risk of overfitting, especially with our augmented dataset, we strategically placed a dropout layer with a rate of 0.25 after each pooling operation.

To prepare the output for final classification, we utilized an adaptive average pooling layer, which gracefully converted the feature maps into a fixed-size 6×6 representation. This step makes the classifier head resilient to minor variations in input size. The flattened output was then fed into our fully connected classifier, which consists of a linear layer with 512 neurons, followed by a ReLU activation and a dropout layer with a rate of 0.5. The final linear layer produced the logits for the 43 traffic sign classes, representing the model’s final prediction.

For training, we used the Cross-Entropy Loss function and optimized the model with the Adam optimizer, setting an initial learning rate of 0.001 and a weight decay of $1e-4$. To ensure the model converged effectively, we implemented a ReduceLROnPlateau scheduler, which would automatically decrease the learning rate by a factor of 0.5 if the validation accuracy stagnated. We trained the model for 30 epochs with a batch size of 16.

B. ResNet-50 with Transfer Learning

To explore the benefits of a more advanced architecture, we chose to use a ResNet-50 model and leverage a transfer learning approach. Instead of training this deep network from scratch, which would be computationally demanding, we took advantage of a ResNet-50 model that was already pre-trained on the vast ImageNet dataset. This decision was motivated by the fact that the pre-trained model had already learned a comprehensive vocabulary of visual features, which we could then fine-tune for our specific task of traffic sign recognition.

We adapted the original ResNet-50 architecture by replacing its final classification layer with a new, custom classifier head designed specifically for the 43 GTSRB classes. The early layers of the ResNet-50 backbone, which are responsible for detecting fundamental visual features, were frozen to preserve their learned knowledge. However, we intentionally unfroze and fine-tuned the later layers (layer3 and layer4), allowing the model to adapt its high-level feature extraction capabilities to the specific visual characteristics of traffic signs.

Our custom classifier head was a multi-layer perceptron built with progressive dimensionality reduction and strong regularization. It consisted of a series of linear layers, each

followed by Batch Normalization, a Gaussian Error Linear Unit (GELU) activation function, and a dropout layer. The final layer was configured to output the logits for the 43 classes.

C. Linear Classifier Baseline

To provide a crucial point of comparison for our deep learning models, we established a performance baseline using a simple linear classifier. This model, essentially a multi-layer perceptron (MLP), represents a non-hierarchical approach to image classification, operating solely on the raw pixel values without any convolutional or spatial awareness.

The model's pipeline began by flattening the input images, converting each $64 \times 64 \times 3$ pixel array into a single vector of 12,288 features. This vector was then passed through a classifier head composed of two hidden linear layers with 512 and 256 neurons, respectively. Each of these layers was followed by a ReLU activation and a dropout layer with a rate of 0.5. The final linear layer provided the predictions for the 43 GTSRB classes.

V. EXPERIMENTAL SETUP

A. Training protocol

We used an identical training procedure for all of our models to ensure that our comparisons were equivalent. We implemented the Adam optimizer to assist in the models' learning and the Cross-Entropy Loss to evaluate how well they performed. We trained for 30 cycles (epochs) using compact batches of 16 the pictures at a time to accelerate up the entire procedure. Based on each model's accuracy on a different validation set, we saved the best version. Additionally, we developed a unique method known as mixed-precision training (AMP) to expedite the procedure without sacrificing quality.

B. Implementation details and reproducibility

We took several kinds of actions to guarantee that our findings could be relied upon and replicated. We fixed the seeds for our Python, NumPy, and PyTorch code libraries, locked down all random elements, and deactivated any potentially unpredictable process steps. In addition, we ensured that the techniques we used to add variations and balance the data (such as targeted augmentation and class-aware sampling) were limited to the training data. This was necessary to avoid any "data leakage" that would unreasonably improve our test and validation set results. Since our dataset was not entirely balanced, we used macro-averaged precision, recall, and F1-scores to measure our results. This provides a broader perspective of performance. To ensure complete transparency, we documented the training times for every model and conducted all of our experiments on a Colab CUDA GPU.

VI. RESULTS

We tested three models: MLP, custom CNN, and ResNet-50. The custom CNN gave the best results with 98.84% accuracy, slightly higher than ResNet-50 at 96.97%, and much better than the MLP baseline at 65.84%. The macro and micro scores

Summary (Test Set, %):

Metric	CNN	MLP	ResNet-50
Accuracy	98.84	65.84	96.97
Precision (Macro)	98.32	64.90	95.14
Recall (Macro)	97.99	65.22	96.71
F1 (Macro)	97.98	63.50	95.79
Precision (Micro)	98.84	65.84	96.97
Recall (Micro)	98.84	65.84	96.97
F1 (Micro)	98.84	65.84	96.97

Fig. 3. Result Summary.

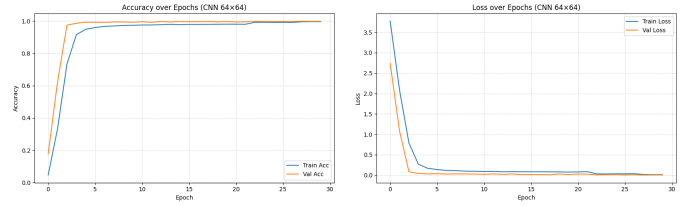


Fig. 4. CNN training curves (accuracy and loss) for 64x64 input.

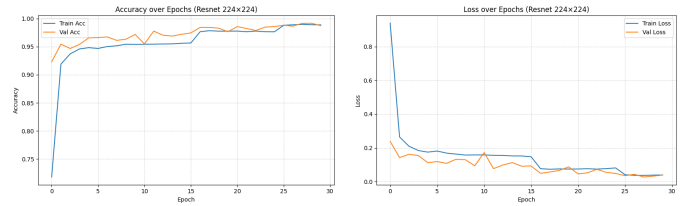


Fig. 5. ResNet-50 training curves (accuracy and loss).

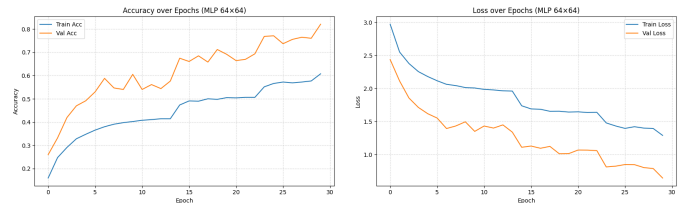


Fig. 6. MLP training curves (accuracy and loss).

for CNN and ResNet-50 were almost the same, meaning they performed well across all classes, not just the common ones.

The MLP struggled because it flattens images and cannot capture spatial patterns, which are very important for recognising traffic signs. Interestingly, the custom CNN slightly outperformed ResNet-50 despite ResNet's transfer learning advantage. This could be because ResNet-50 was only partially fine-tuned, the domain gap from ImageNet, and the CNN being specifically designed for 64x64 inputs with strong regularisation for this task.

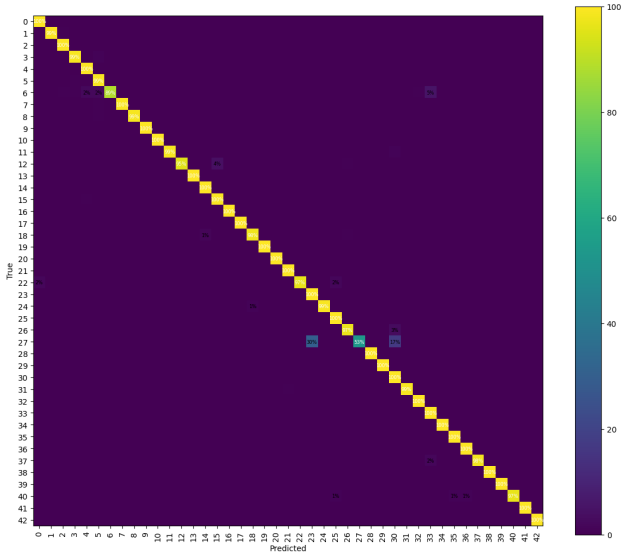


Fig. 7. CNN confusion matrix on the GTSRB test set.

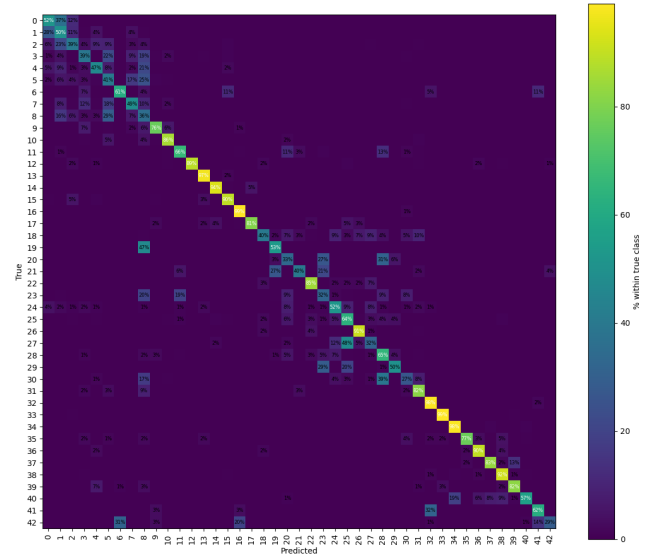


Fig. 9. MLP confusion matrix on the GTSRB test set.

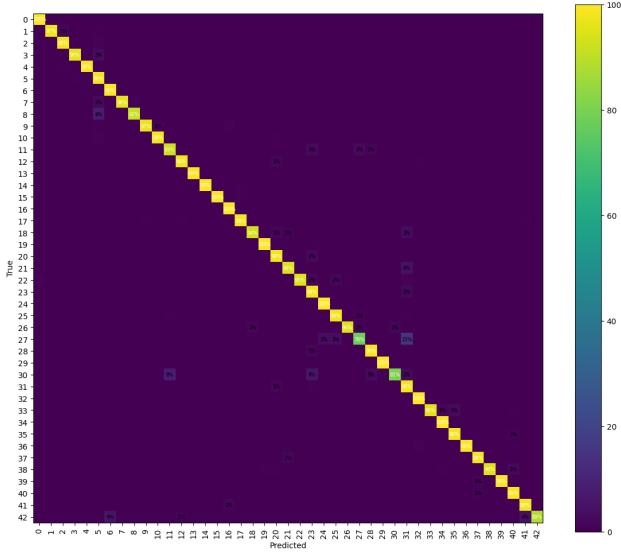


Fig. 8. ResNet-50 confusion matrix on the GTSRB test set.

VII. CONCLUSION

The study shows that a carefully made model can be both accurate and fast for traffic sign recognition. Deep learning models gave much better results than the MLP, and the custom CNN scored slightly higher than ResNet-50. A model built for the task can sometimes work better than a large model trained earlier on other data, which is useful for driverless cars where speed and accuracy are both important.

Preparing the data well also played a big role. Balancing the number of images in each class, adding variations through augmentation, and splitting the dataset properly helped the models learn from many different examples instead of only the common signs. Most mistakes were with signs that looked almost the same or with images of poor quality. Future work

should focus on telling apart very similar signs and dealing with unclear images to make traffic sign recognition more reliable in real driving conditions.

REFERENCES

- [1] N. Dalal and B. Triggs, "Histograms of Oriented Gradients for Human Detection," in *Proc. IEEE Computer Society Conf. Computer Vision and Pattern Recognition (CVPR)*, vol. 1, pp. 886–893, 2005, doi: 10.1109/CVPR.2005.177.
- [2] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, pp. 770–778, 2016, doi: 10.1109/CVPR.2016.90.
- [3] J. Stallkamp, M. Schlipsing, J. Salmen, and C. Igel, "The German Traffic Sign Recognition Benchmark: A Multi-Class Classification Competition," in *Proc. IEEE Int. Joint Conf. Neural Netw. (IJCNN)*, pp. 1453–1460, 2011, doi: 10.1109/IJCNN.2011.6033395.
- [4] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet Classification with Deep Convolutional Neural Networks," in *Proc. Advances in Neural Information Processing Systems (NIPS)*, vol. 25, 2012.
- [5] D. C. Ciresan, U. Meier, J. Masci, L. M. Gambardella, and J. Schmidhuber, "Multi-column Deep Neural Network for Traffic Sign Classification," *Neural Networks*, vol. 32, pp. 333–338, 2012, doi: 10.1016/j.neunet.2012.02.023.
- [6] G. Huang, Z. Liu, L. Van Der Maaten, and K. Q. Weinberger, "Densely Connected Convolutional Networks," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, pp. 4700–4708, 2017, doi: 10.1109/CVPR.2017.243.
- [7] A. Dosovitskiy et al., "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2021. [Online]. Available: <https://arxiv.org/abs/2010.11929>
- [8] X. Wang, R. Girshick, A. Gupta, and K. He, "Non-local Neural Networks," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, pp. 7794–7803, 2018, doi: 10.1109/CVPR.2018.00813.
- [9] A. Antoniou, A. Storkey, and H. Edwards, "Data Augmentation Generative Adversarial Networks," *arXiv preprint*, 2017. [Online]. Available: <https://arxiv.org/abs/1711.04340>
- [10] S. Kundu, S. Banerjee, and A. K. Pal, "Traffic Sign Recognition Using Hybrid CNN–Transformer Architecture," *IEEE Access*, vol. 10, pp. 125871–125884, 2022. [Online]. Available: <https://doi.org/10.1109/ACCESS.2022.3224567>